AEE.Bitácora III

Docente: Willman Acosta Lugo

Fecha: 17-04-2026

Tabla de Contenido:

1. El Escenario: "La Defensa en Profundidad y el rastro digital"....1
2. Fase de Investigación: Comprendiendo el corazón del sistema.... 1
3. Fase Práctica: Hardening y Vigilancia en la Trinchera....2
Reto A: Auditoría de Accesos e Intrusiones (Log Analysis y Scripting).... 2
Reto B: Configuración del "Escudo" Perimetral con UFW....3
4. Rúbrica de Evaluación Detallada y Feedback....3
Referencias....4

1. El Escenario: "La Defensa en Profundidad y el rastro digital"

En la administración de sistemas moderna, la invisibilidad es una de las mejores defensas, pero nunca debe ser la única. Hablamos constantemente del concepto de **Defensa en Profundidad (Defense in Depth)**. Esto significa que no podemos confiar únicamente en una contraseña fuerte o en un cortafuegos; necesitamos múltiples capas superpuestas. Si un atacante logra traspasar la primera barrera, debe encontrarse con una segunda, y luego con una tercera. Y lo que es más importante: si logra encontrar la puerta, vuestro servidor debe ser capaz de registrar y alertar de cada uno de sus movimientos.

Y es que, para que os hagáis una idea, un servidor Linux recién expuesto en la nube (ya sea en AWS, Azure o GC) tarda, de media, menos de 5 minutos en recibir su primer escaneo de red automatizado. Existen redes enteras de bots rastreando Internet 24/7 en busca de puertos SSH abiertos y vulnerabilidades conocidas.

Ya no serás un usuario convencional; actuarás como **Administrador de Sistemas (SysAdmin)** en medio de un despliegue crítico y con el reloj en contra. El objetivo es claro: fortificar el sistema mediante técnicas de *Hardening*, levantar un muro de fuego *stateful* (*Firewall*) y automatizar la vigilancia de los registros antes de que acabe vuestro turno.

Recordad el pilar fundamental de los sistemas basados en Unix: "**Todo es un fichero**". Desde la tarjeta de red virtual, pasando por la memoria RAM, hasta llegar a la configuración del firewall y los registros de seguridad. Entender esta arquitectura es la llave maestra para dominar cualquier servidor.

2. Fase de Investigación: Comprendiendo el corazón del sistema

Antes de realizar cambios a lo loco en un entorno de producción, un buen administrador debe investigar y fundamentar sus decisiones. La improvisación técnica suele derivar en fallos de seguridad críticos o en servidores inoperativos. Utilizad vuestro acceso a las bases de datos académicas [1] para dotar de base científica y metodológica a vuestro trabajo:

- **Reto de Investigación 1**.
  - ¿Qué información específica (como PIDs, nombres de usuario o direcciones IP) diferencia un intento fallido de conexión remota *SSH* de un simple fallo de contraseña de un usuario local frente a la pantalla?
- **Reto de Investigación 2**. Cumplimiento y Log Management. Busca en **Dialnet** o **Semantic Scholar** artículos sobre *Log Management* (Gestión centralizada de registros). A nivel empresarial y legal (piensa en el RGPD en España), ¿qué ventajas vitales ofrece enviar y custodiar los logs en un servidor externo seguro en

lugar de mantenerlos dispersos e indefensos en la propia máquina que podría ser vulnerada?

- **Documentación obligatoria**. Crea un archivo **README.md** en la raíz de vuestro repositorio GitHub. La primera sección debe titularse "Fundamentos de Seguridad y Auditoría". En ella, debéis explicar todos estos conceptos analizados con vuestras propias palabras (nada de copiar y pegar). Por supuesto, debéis respaldar vuestras afirmaciones citando al menos dos fuentes reales y accesibles siguiendo estrictamente el manual de estilo IEEE [2].

REALIZADA EN EL DOCUMENTO MD.

3. Fase Práctica: *Hardening* y Vigilancia en la Trinchera

Esta fase debe ser documentada paso a paso. En una auditoría real, no basta con mostrar el resultado final; el "cómo se llegó ahí", las decisiones tomadas y los comandos ejecutados son la verdadera evidencia de vuestra competencia técnica.

Reto A: Auditoría de Accesos e Intrusiones (Log Analysis y Scripting)

Primero, intentamos entrar con un usuario falso en el equipo. Vemos ipconfig para saber nuestra IP e intentar entrar. (EJEMPLO DE CAPTURA CON 1 SOLO)

```
C:\Users\DAM1>ssh administrador_falso@172.22.0.170
ssh: connect to host 172.22.0.170 port 22: Connection timed out
```

En segundo lugar, vamos a ver quién ha entrado o intentado entrar usando un sudo grep que busque la palabra Failed password en /var/log. usamos el grep para que nos busque texto.

```
moreno@alejandro:~$ sudo grep "Failed password" /var/log/auth.log
Apr 17 08:04:14 alejandro sudo:   moreno : TTY=tty1 ; PWD=/home/moreno ; USER=root ; COMMAND=/usr/bi
n/grep Failed password /var/log/auth.log
moreno@alejandro:~$ _
```

Para proseguir, Como imaginaréis, un administrador no puede pasarse la mañana leyendo miles de líneas de texto plano por ello vamos a crear un script que automatice esto.

Primero creamos una carpeta con mkdir -p para crear la estructura completa. en home llamada scripts en la que al meternos con cd creamos un archivo con nano llamado de esta manera:
nano check_intruders.sh

Reto B: Configuración del "Escudo" Perimetral con UFW

Bajo el capó de Ubuntu trabaja el potente motor de filtrado de paquetes *iptables* (o *nftables*). Sin embargo, nosotros utilizaremos **UFW (Uncomplicated Firewall)**, una interfaz amigable que nos facilita la vida. Vamos a aplicar la arquitectura más segura: una política de "Denegación por Defecto". Todo lo que no esté explícitamente invitado a entrar, se quedará fuera.

1. **Reglas de Sigilo** (*Drop* vs *Reject*). Configura el firewall para que el servidor ignore completamente las peticiones ICMP Echo Request (el famoso comando ping). Un servidor que hace drop (dejar caer el paquete sin responder) a los pings, se vuelve prácticamente invisible frente a los escaneos de red superficiales que buscan hosts vivos.
2. **Reglas de Servicio**. Por otro lado, si bloqueamos todo, nosotros tampoco podremos administrarlo. Permitid el acceso TCP exclusivamente al puerto 22 (SSH). Si, como buena práctica, habéis cambiado el puerto SSH por defecto a uno superior, aseguraos de abrir ese puerto específico.
3. **Activación y Evidencias**. Ejecuta *sudo ufw enable*. Posteriormente, verifica el estado y la carga real de las reglas con los comandos *sudo ufw status numbered* y *sudo ufw status verbose*.
4. **Prueba cruzada**. En vuestra documentación, debéis incluir capturas demostrando que desde la máquina anfitriona el *ping* da error de tiempo de espera, pero la conexión por cliente SSH (como Windows PowerShell o el Terminal) sí se establece correctamente.

4. Rúbrica de Evaluación Detallada y Feedback

Para que no haya sorpresas, la calificación de esta sesión se otorgará estrictamente en base al cumplimiento de los criterios descritos en la siguiente matriz. Además, durante los últimos 15 minutos de la clase, realizaremos una rápida inspección cruzada (Peer Review) donde comprobaréis la robustez del servidor de vuestro compañero de mesa.

Referencias